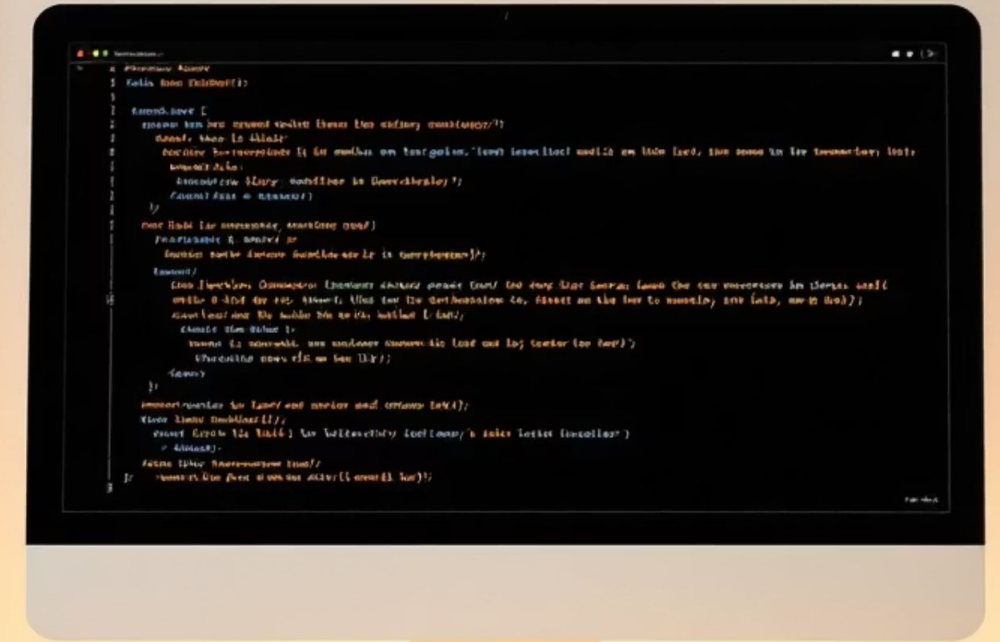


CODE REVIEWER

A full-stack **multi-agent LLM-powered code reviewer**, fix generator, evaluator, and GitHub PR automation system that demonstrates an end-to-end **AI + Cloud MLOps pipeline**. The system intelligently reviews code, detects bugs and security vulnerabilities, generates production-ready fixes, evaluates solution quality, and integrates directly with GitHub workflows.



Technology Stack

Core Infrastructure

Amazon Bedrock (Claude Sonnet): LLM inference engine

AWS RDS PostgreSQL: persistent session & metrics storage

AWS S3: fixed-code artifact repository

AWS CloudWatch Logs: comprehensive AgentOps logging

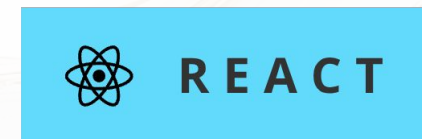
Application Layer

Next.js + React: responsive UI framework

Node.js + Express: RESTful API backend

Prisma ORM: type-safe database access

GitHub OAuth + Octokit: automated PR creation



System Architecture

1

Frontend Layer

Next.js/React UI handles user interactions and displays review results in real-time

2

API Gateway

Node.js + Express backend orchestrates requests and coordinates agent workflows

3

Multi-Agent Core

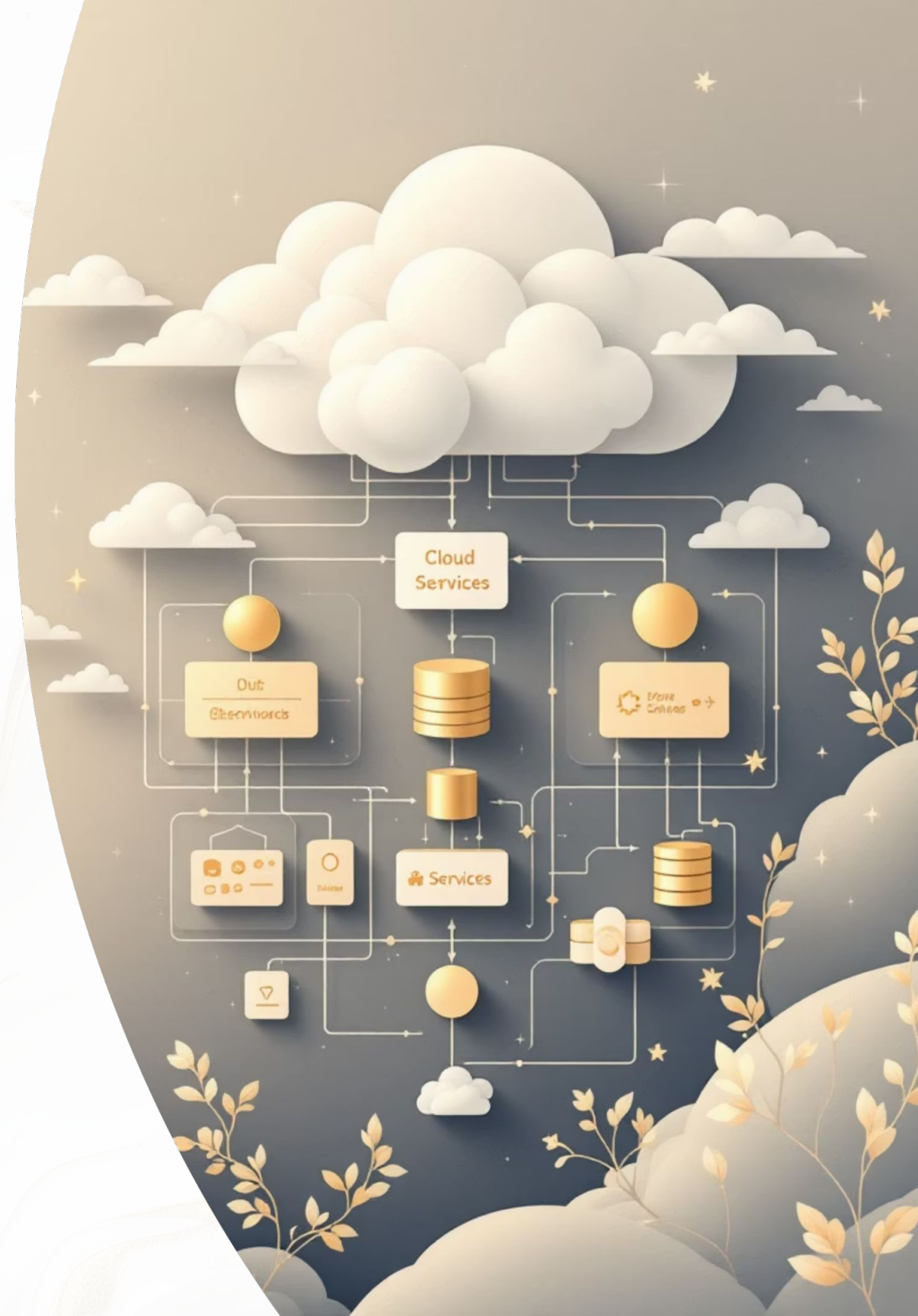
Six specialized agents analyze code through Bedrock LLM with intelligent merging

4

Cloud Storage

RDS stores sessions, S3 archives artifacts, CloudWatch captures telemetry

The system processes code through a sequential pipeline where each agent builds upon previous findings, ensuring comprehensive analysis from syntax to security.



Repository Structure

Application Layer

```
apps/  
├── api/                # Express backend  
│   ├── src/index.ts  
│   ├── src/s3.ts  
│   └── src/parseGithubUrl.ts  
└── web/                # Next.js frontend  
    ├── app/local  
    ├── app/upload  
    ├── app/github  
    └── components/
```

Shared Packages

```
packages/  
├── agents/            # Multi-agent logic  
├── types/             # Review interfaces  
├── db/                # Prisma + RDS  
└── github/            # GitHub connector  
  
Root:  
├── docs/  
├── package.json  
└── pnpm-workspace.yaml
```

The monorepo architecture uses **pnpm workspaces** for efficient dependency management, enabling shared type definitions and utilities across frontend, backend, and agent packages while maintaining clean separation of concerns.

Multi-Agent Pipeline

StaticAgent

Pattern-based detection of TODOs, unused imports, and code smells

SecurityAgent

Identifies SQL injection, hardcoded secrets, and unsafe logging practices

FixAgent

Generates production-ready rewrites with contextual improvements

ReviewAgent

Deep semantic analysis via LLM for architectural and design issues

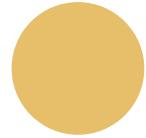
BugDetectionAgent

Finds logic errors, null pointer risks, and exception handling gaps

EvalAgent

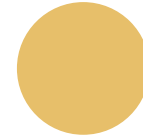
Scores correctness, assesses risk levels, and summarizes enhancements

Key Features



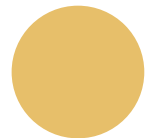
Three Input Modes

- Local editor for quick paste reviews
- File upload for batch processing
- GitHub integration with OAuth authentication



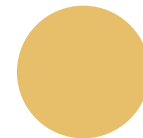
Automated Fix Generation

- Context-aware code rewriting
- Side-by-side comparison view
- Downloadable fixed artifacts



GitHub Integration

- OAuth-based authentication
- Repository and branch navigation
- Automated PR creation from reviews



Cloud MLOps

- PostgreSQL session persistence
- S3 artifact versioning
- CloudWatch operational insights

System in Action

GitHub Review Interface

OAuth-authenticated repository
browsing with branch and file selection
for seamless code review initiation

Multi-Agent Output

Comprehensive analysis with
categorized findings, severity ratings,
and actionable recommendations

Cloud Artifacts

Versioned storage of fixed code in S3
with metadata tracking and
CloudWatch logging integration

Installation & Setup

1

Backend Setup

```
cd apps/api  
pnpm install  
pnpm dev
```

Starts Express server on default port
with hot reload enabled

2

Frontend Setup

```
cd apps/web  
pnpm install  
pnpm dev
```

Launches Next.js development server
with React Fast Refresh

3

Database Migration

```
cd packages/db  
pnpm --filter  
@code-reviewer/db  
prisma migrate dev
```

Applies Prisma schema migrations to
RDS PostgreSQL instance

Prerequisites: Configure AWS credentials, GitHub OAuth app, and database connection strings in `.env` before running setup commands. See `.env.example` for required variables.

Conclusion

This project showcases a production-grade AI-powered code review platform that seamlessly integrates cutting-edge LLM capabilities with enterprise cloud infrastructure. By combining multi-agent reasoning, automated fix generation, and GitHub workflow automation, it demonstrates a complete MLOps pipeline ready for real-world software engineering teams.

6

Specialized Agents

Comprehensive analysis pipeline

3

Input Modes

Flexible integration options

100%

Cloud-Native

AWS infrastructure foundation